

COMP2054-ADE

Lec16

Map ADT : using lists

Abstract Data Type: Map

- A map stores a collection of key-value
 - In this respect, it is similar to a Priority Queue PQ
- The primary difference is that a Map allows to search for entries by their key
 - In contrast a PQ only has to support access/removal of the minimum key
 - A map also allows to remove entries by their key
 - Of course, it also allows to insert/add new entries
- Typically, and in this module, multiple entries in a Map with the same key **are NOT** allowed
 - Inserting an entry when inserting an existing key, will overwrite the old key-value pair
 - Multiple copies of a key are allowed in a 'multi-map'
 - Note: Multiple copies of a key **are** allowed in a PQ

Maps: Usage

- Maps are very useful and common in many coding tasks
 - Especially, when we want something similar to a data base
- E.g.
 - Storing employee/student data by their employee/student ID
 - Address books, with name as the key
 - Symbol table in compilers
- May be known as “dictionary” or “associative arrays” (perl, php)
 - because they associates some values with a key, and the key need not be an integer.

VERY IMPORTANT

We will see that Maps can be implemented using special binary trees, called “Binary Search Trees”

Be careful not to confuse different usages of binary trees:

- Priority Queue &
“Binary Tree implementations of heaps”

with

- MAP &
“Binary Search Trees”

Both insert entries based on a key.

The crucial difference is the way entries are accessed:

- MAP: access **any** key
- PQ: access only the **minimum** key

These different requirements lead to different conditions on the trees, and different algorithms

Map – main operations

Main operations are

- V get(K k)
 - Gets the value associated with the key k , or null (or similar) if the key k is not in the map
 - Does not change the contents of the map
- (K,V) add(K k,V v)
 - add the Key-Value pair (k,v) to the storage
 - If the key is already in the map, then it will be overwritten to prevent multiple keys
 - The existing key-value pair may be returned
- V remove(K k)
 - Gets the value associated with the key k , or null (or similar) if the key k is not in the map
 - Also, removes the key k entry

Map – other operations

Also typically have

- `int size()`
 - Returns the number of Key-Value pairs stored
- And others such as
 - `(K,V) GetMin()`
 - Returns the minimum value Key `k` in the current map
 - etc

A Simple List-Based Map

- It is straightforward, but not efficient, to implement a map using a list – either singly or doubly-linked.
- We also store separate size counter, n , so that “size()” is $O(1)$, and maintain it on adding or removing elements

Map: using lists/arrays

- As in many cases, we can do a first implementation using lists or arrays
- If the list is unsorted:
 - `Add(int k)` would be $O(1)$ as we can just put the entry at the start. BUT we must overwrite any values if it is already in the list, hence must walk and it is $O(n)$ (see later slide)
 - `remove(int k)` is $O(n)$ as we must scan the list for the key
- If the list is sorted (smaller first):
 - `add(k)` is $O(n)$ as must scan to find the correct position to insert it

The get(k) Algorithm

Simply

1. Walk along the list looking for key k
2. If find it then return the currently associated value
3. Else return null (or false)

Worst case: $O(n)$

The put(k,v) Algorithm

Simply

1. Walk along the list looking for key k
2. If find it then return the currently associated value, and overwrite with the new value v
3. Else insert $\langle k,v \rangle$ as a node at the end of the list

Note: we cannot just put the new entry at the start of the list, because we need to check that the key k is not already stored !

Worst case: $O(n)$

The remove(k) Algorithm

Simply

1. Walk along the list looking for key k
2. If find it then return the currently associated value and remove the node.
3. Else return null/false

Worst case: $O(n)$

List-Based Map

- Overall: the operations are $O(n)$ because of needing to traverse the list.
- The unsorted list implementation is suitable only for maps of small size
- We could use a sorted list, but still $O(n)$
 - It does not help as we need to be able to access any k , not just the minimum

Aim:

- Can we find an implementation of a Map such that **all** (or most) operations are (usually) better than $O(n)$?
 - I.e. ideally will all operations $o(n)$.
 - E.g. aim for them all to be $O(\log n)$
- We will study two ways to implement Maps:
 - Hash Maps
 - Binary Search Trees

Expectations

- Know and understand the Map ADT
- Be clear as to the similarities and differences from the Priority Queue ADT
- Understand how to do simple array or list implementations